

JavaScript Study Notes

Arrays & Nested Loops · String Methods · Math Methods · Mini Project: Number Guessing Game

These notes cover the concepts practiced in your app.js file: looping over arrays (including nested/2D arrays), searching arrays for a value, common string methods, common Math methods, and how they all come together in a small number-guessing game.

1. Arrays & Nested Loops

1.1 Looping over a simple array

A **for loop** repeats code a set number of times. When you loop over an array, the loop variable (commonly *i*) represents the index — the position of each item, starting at 0.

```
var arr = ["apple", "mango", "banana"];

for (var i = 0; i < arr.length; i++) {
  console.log(arr[i]);
}
// Output: apple, mango, banana
```

1.2 Nested loops

A nested loop is a loop inside another loop. The **inner loop** completes all of its cycles for every single cycle of the **outer loop**. In your code, the outer loop picks each fruit, and the inner loop (running 3 times) prints that same fruit 3 times before moving to the next one.

```
for (var i = 0; i < arr.length; i++) {
  for (var j = 0; j < 3; j++) {
    console.log(arr[i]);
  }
}
// Prints each fruit name 3 times in a row
```

1.3 Nested arrays (2D arrays) & searching with a flag

A nested array is an array whose items are themselves arrays — useful for grid-like or grouped data. To search inside it, you need two indices: one for the outer array (*i*) and one for the inner array (*j*). A **flag** variable (true/false) is a common pattern to remember whether something was found after the loop finishes.

```
var li = [[1, 2, 3], ["a", "b", "c"]];
var userInput = prompt("Enter anything");
var flag = false;

for (var i = 0; i < li.length; i++) {
  for (var j = 0; j < 3; j++) {
    if (li[i][j] == userInput) {
      flag = true;
      console.log("Found");
    }
  }
}
```

```

        break;    // exits only the INNER loop
    }
}
if (!flag) {
    console.log("not found");
}

```

Key point: break only exits the loop it is directly inside. In a nested loop, break stops the inner loop and lets the outer loop continue to its next value.

1.4 Case-insensitive search example

When comparing user input to array values, differences in capitalization can cause a match to fail.

.toLowerCase() converts both sides to lowercase before comparing, so "Karachi" and "karachi" are treated as equal.

```

var city = ["karachi", "lahore", "islamabad", "multan", "pindi"];
var userInput = prompt("Enter your city name: ");
var flag = false;

for (var i = 0; i < city.length; i++) {
    if (city[i].toLowerCase() === userInput.toLowerCase()) {
        console.log(userInput + " city found");
        flag = true;
        break;
    }
}
if (!flag) {
    console.log(userInput + " city not found");
}

```

2. String Methods

Strings have built-in methods for reading, searching, and transforming text. Below are the ones used in your code, each demonstrated on the string "Hello world".

Method	What it does	Example	Result
<code>.length</code>	Property (no parentheses) that gives the number of characters	<code>txt.length</code>	11
<code>.slice(start, end)</code>	Extracts characters from index start up to (not including) end	<code>txt.slice(6, 9)</code>	"wor"
<code>.indexOf(sub)</code>	Returns the index where a substring first appears, or -1 if not found	<code>txt.indexOf("wor")</code>	6
<code>.charAt(index)</code>	Returns the character at a specific index	<code>txt.charAt(0)</code>	"H"
<code>.replace(old, new)</code>	Returns a new string with the first match of old text swapped for new text	<code>txt.replace("apple", "orange")</code>	"this is an orange"

Important: `.replace()` does not change the original string — strings are immutable in JavaScript. It returns a brand new string, which is why you must reassign it back: `txt = txt.replace(...)`.

```
var txt = "Hello world";
console.log(txt.length);           // 11
console.log(txt.slice(6, 9));      // "wor"
console.log(txt.indexOf("wor"));   // 6
console.log(txt.charAt(0));        // "H"

var txt2 = "this is an apple";
txt2 = txt2.replace("apple", "orange");
console.log(txt2);                // "this is an orange"
```

3. Math Methods

The built-in Math object provides methods for rounding and generating random numbers. These are used with `num = 79.54` below.

Method	Behavior	Math.__(79.54)
<code>Math.round(x)</code>	Rounds to the nearest whole number (.5 rounds up)	80
<code>Math.floor(x)</code>	Always rounds DOWN, toward negative infinity	79
<code>Math.ceil(x)</code>	Always rounds UP, toward positive infinity	80

Math.random()	Returns a random decimal between 0 (inclusive) and 1 (exclusive)	e.g. 0.372...
----------------------	--	---------------

Generating a random whole number between 1 and 10: `Math.random()` alone gives a decimal like 0.83. Multiplying by 10 stretches that range to 0–10 (e.g. 8.3). `Math.ceil()` then rounds that up to a whole number from 1 to 10 (0 only occurs if `random()` returns exactly 0, which rounds up to 0 in rare edge cases — in practice this pattern is common but `Math.floor(Math.random()*10)+1` is the more precise, commonly taught version).

```
var cN = Math.ceil(Math.random() * 10);  
// cN is a whole number, generally between 1 and 10
```

4. Mini Project: Number Guessing Game

This project combines random numbers, prompts, type conversion, and conditionals into one small game.

4.1 The unary plus (+) operator

`prompt()` always returns a **string**, even if the user types a number. Putting a `+` directly in front of a variable or expression converts it to a **number**. This matters because `"5" === 5` is false (different types), but `+"5" === 5` is true.

```
var userInput = +prompt("Guess any number between 1 to 10");  
// The leading + converts the typed string into a number
```

4.2 Full code, explained

```
var cN = Math.ceil(Math.random() * 10);  
var userInput = +prompt("Guess any number between 1 to 10", cN);  
  
if (userInput === cN) {  
    console.log("Yes you guessed it right");  
} else if (cN - 1 === userInput || cN + 1 === userInput) {  
    console.log("So Close");  
} else {  
    console.log("You lose");  
}
```

Step by step:

- `cN` is a randomly generated "correct number" from 1–10.
- `prompt()`'s second argument is a default value shown in the input box.
- `===` checks both value AND type — the unary `+` is what makes this comparison work correctly.
- The `||` (logical OR) means the "So Close" branch runs if EITHER side is true — i.e. the guess is exactly one number below or one number above the correct answer.
- If neither condition matches, the `else` branch runs.

Logical operators reminder: `&&` (AND) requires both sides to be true; `||` (OR) requires at least one side to be true; `!` (NOT) flips a boolean, which is why `if (!flag)` means "if flag is false."

Next: keep practicing conditionals + arrays together — try rewriting the city-search example to search a 2D array of [city, country] pairs.